CyberSentinel AI

Beginner-to-Portfolio Guide for an AI Cyberattack Detection System

Machine Learning + Cybersecurity + FastAPI + Dashboard + TensorFlow

Purpose: Build a defensive intrusion-detection project from zero, train and compare ML models, deploy the best model through an API, and present results in a dashboard.

Scope: This guide uses public/controlled datasets for defensive research. It does not require attacking real systems or collecting unauthorized traffic.

1. What You Will Build

You will build a system that reads network-flow features and predicts whether the traffic is normal or malicious. After the first version works, you will extend it to predict specific attack classes and detect unusual behavior using an autoencoder.

```
Network Dataset / Flow Features
        |
        v
Data Cleaning + Preprocessing
        |
        v
Train Multiple Models
(Logistic Regression / Random Forest / XGBoost / Neural Network)
        |
        v
Evaluate and Select Best Model
        |
        v
Save Model + Preprocessing Objects
        |
        v
FastAPI Prediction Service
        |
        v
Dashboard / Alerts
```

Final portfolio features

- Binary classification: Normal vs Attack.
- Multiclass classification: Normal, DDoS, brute force, port scan, botnet, and other labels available in your dataset.
- Model comparison using accuracy, precision, recall, F1-score, ROC-AUC where appropriate, and confusion matrix.
- Feature importance or SHAP explanations.
- FastAPI REST endpoint for predictions.
- Frontend/dashboard for security alerts and statistics.
- Optional TensorFlow autoencoder for anomaly detection.

2. What “Training a Model” Means

Training means giving an algorithm many examples where both the input features and the correct answer are known. The algorithm adjusts internal parameters so that it can predict the answer for new data that it has not seen before.

```
Training example:
X = [flow duration, packets/sec, bytes/sec, port, ...]
y = Attack

model.fit(X_train, y_train)
```

```
After training:
new_network_flow -> model -> Normal or Attack
```

Key idea: You are not manually writing rules such as “if packets > 1000, call it an attack.” The algorithm learns patterns from labeled examples.

3. Recommended Technology Stack

Part	Recommended tools
Language	Python 3.11 or 3.12
Data	Pandas, NumPy
ML	Scikit-learn, XGBoost
Deep learning	TensorFlow / Keras
Plots	Matplotlib
Model saving	joblib
Backend	FastAPI + Uvicorn
Frontend	React or Streamlit
Database (optional)	PostgreSQL
Deployment	Docker + a cloud host

4. Create the Project Environment

4.1 Folder structure

```
cyber-sentinel-ai/
|-- data/
|-- notebooks/
|   |-- 01_exploration.ipynb
|   |-- 02_preprocessing.ipynb
|   |-- 03_logistic_regression.ipynb
|   |-- 04_random_forest.ipynb
|   |-- 05_xgboost.ipynb
|   `-- 06_neural_network.ipynb
|-- src/
|   |-- preprocessing.py
|   |-- train.py
|   `-- evaluate.py
|-- models/
|-- backend/
|   `-- main.py
|-- frontend/
|-- requirements.txt
`-- README.md
```

4.2 Create and activate a virtual environment

```
python -m venv venv

# macOS / Linux
source venv/bin/activate
```

```
# Windows PowerShell
venv\Scripts\Activate.ps1
```

A virtual environment keeps the packages for this project separate from packages used by other Python projects.

4.3 Install packages

```
pip install pandas numpy scikit-learn matplotlib joblib jupyter
pip install xgboost tensorflow fastapi uvicorn pydantic

pip freeze > requirements.txt
```

The second command saves the installed package versions so another person can reproduce your environment.

5. Choose and Prepare a Dataset

Use a public intrusion-detection dataset such as CICIDS2017, UNSW-NB15, or NSL-KDD. Start with one dataset only. The exact column names differ between datasets, so inspect the data before writing preprocessing code.

Beginner recommendation: Use a manageable subset first. A smaller clean experiment is better than trying to load millions of rows and getting stuck because of memory.

5.1 Load a CSV file

```
import pandas as pd

df = pd.read_csv("data/network_data.csv")

print(df.head())
print(df.shape)
print(df.columns.tolist())
print(df.dtypes)
```

Explanation: `read_csv` loads the file into a DataFrame. `head()` shows example rows. `shape` shows row and column counts. `dtypes` shows whether each column is numeric or text.

5.2 Inspect class labels

```
print(df["Label"].value_counts())
print(df["Label"].value_counts(normalize=True))
```

The first line shows how many examples exist for each class. The second shows percentages. This tells you whether the dataset is imbalanced.

6. Data Cleaning

6.1 Standardize column names

```
df.columns = (
  df.columns
```

```
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)

print(df.columns[:10])
```

This makes columns easier to reference consistently. For example, `Flow Duration` becomes `flow_duration`.

6.2 Remove duplicate rows

```
before = len(df)
df = df.drop_duplicates()
after = len(df)
print("Removed duplicates:", before - after)
```

Duplicate rows can bias a model because the same example is counted multiple times.

6.3 Handle infinite and missing values

```
import numpy as np

df = df.replace([np.inf, -np.inf], np.nan)
print(df.isna().sum().sort_values(ascending=False).head(20))

df = df.dropna()
```

Some network-rate features may contain infinity because of division by zero. Replace infinity with missing values, inspect them, and for a first project remove rows with unresolved missing data.

7. Create the First Target: Normal vs Attack

Begin with binary classification. It is easier to debug and helps you learn the full ML pipeline.

```
# Change the label column name if your dataset uses a different name.
NORMAL_NAMES = {"benign", "normal"}

df["target"] = df["label"].astype(str).str.strip().str.lower().apply(
   lambda label: 0 if label in NORMAL_NAMES else 1
)

print(df["target"].value_counts())
```

Explanation: normal/benign rows become `0`; every malicious class becomes `1`. Always verify the label spelling in your chosen dataset before applying this mapping.

8. Separate Features (X) and Target (y)

```
# Remove the original text label and the target from the feature set.
X = df.drop(columns=["label", "target"])
y = df["target"]

print(X.shape)
print(y.shape)
```

`X` contains information the model is allowed to use. `y` is the correct answer. Never leave the target or an equivalent label column inside `X`, because that creates data leakage.

8.1 Keep numeric columns for the first model

```
X = X.select_dtypes(include=["number"])
print("Numeric feature count:", X.shape[1])
```

Many intrusion datasets are mostly numeric. Starting with numeric features avoids categorical-encoding complexity while you learn the pipeline.

9. Split Into Training and Test Data

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training:", X_train.shape)
print("Testing:", X_test.shape)
```

`test_size=0.20` reserves 20% of the data for final testing. `stratify=y` tries to preserve the class ratio. `random_state=42` makes the split reproducible.

Important: Never fit preprocessing tools on the full dataset before the split. Fit them using the training set, then apply them to validation/test data.

10. Create Reusable Evaluation Code

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

def evaluate_binary_model(model, X_test, y_test):
    predictions = model.predict(X_test)

    print("Accuracy :", accuracy_score(y_test, predictions))
    print("Precision:", precision_score(y_test, predictions))
    print("Recall   :", recall_score(y_test, predictions))
    print("F1-score :", f1_score(y_test, predictions))
    print("\nConfusion matrix:\n", confusion_matrix(y_test, predictions))
    print("\nClassification report:\n")
```

```
    print(classification_report(y_test, predictions, digits=4))

    return predictions
```

Accuracy is useful but can be misleading when normal traffic heavily outnumbers attacks. Precision measures false alarms. Recall measures how many actual attacks you detect. F1 balances precision and recall.

11. Model 1 - Logistic Regression

Use Logistic Regression as a baseline. It is simple, fast, and gives you a reference point for stronger models.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

logistic_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=2000, class_weight="balanced"))
])

logistic_model.fit(X_train, y_train)
logistic_predictions = evaluate_binary_model(
    logistic_model, X_test, y_test
)
```

The pipeline scales features and then trains Logistic Regression. `class_weight="balanced"` can help when the classes are uneven, but it is not a substitute for careful imbalance analysis.

12. Model 2 - Random Forest

Random Forest is often a strong choice for tabular network-flow features because it can model nonlinear interactions and does not require feature scaling.

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)
rf_predictions = evaluate_binary_model(
    rf_model, X_test, y_test
)
```

`n_estimators=200` builds 200 trees. `n_jobs=-1` uses available CPU cores. `class_weight="balanced"` tells the classifier to pay more attention to minority classes based on their frequency.

12.1 Feature importance

```
import pandas as pd

feature_importance = pd.DataFrame({
    "feature": X_train.columns,
    "importance": rf_model.feature_importances_
}).sort_values("importance", ascending=False)

print(feature_importance.head(20))
```

This produces a ranking of features used by the forest. Importance does not prove causation, but it is useful for model interpretation and debugging.

13. Model 3 - XGBoost

XGBoost builds trees sequentially, with later trees focusing on mistakes made by earlier trees. It is a common high-performance model for structured/tabular data.

```
from xgboost import XGBClassifier

# For very imbalanced data, calculate a scale_pos_weight from the training set.
negative = (y_train == 0).sum()
positive = (y_train == 1).sum()
scale_pos_weight = negative / positive

xgb_model = XGBClassifier(
    n_estimators=300,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=scale_pos_weight,
    eval_metric="logloss",
    random_state=42,
    n_jobs=-1
)

xgb_model.fit(X_train, y_train)
xgb_predictions = evaluate_binary_model(
    xgb_model, X_test, y_test
)
```

The imbalance weight is computed only from the training labels. Start with these values, then tune them later using validation or cross-validation instead of guessing endlessly.

14. Model 4 - TensorFlow Neural Network

14.1 Scale the data

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

The scaler learns the training-set mean and standard deviation. The test set is transformed using the already-fitted scaler to prevent leakage.

14.2 Build the network

```
import tensorflow as tf

tf.random.set_seed(42)

nn_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train_scaled.shape[1],)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.30),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dropout(0.20),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

nn_model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=[
        "accuracy",
        tf.keras.metrics.Precision(name="precision"),
        tf.keras.metrics.Recall(name="recall")
    ]
)

nn_model.summary()
```

The output uses `sigmoid`, producing a probability between 0 and 1. Binary cross-entropy is the standard loss for binary classification.

14.3 Train the neural network

```
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)

history = nn_model.fit(
    X_train_scaled,
    y_train,
    validation_split=0.20,
    epochs=30,
    batch_size=128,
    callbacks=[early_stopping],
    verbose=1
)
```

An epoch is one pass through the training data. `validation_split=0.20` holds back part of the training data for validation. Early stopping stops training if validation loss stops improving.

14.4 Evaluate the neural network

```
import numpy as np

probabilities = nn_model.predict(X_test_scaled).ravel()
nn_predictions = (probabilities >= 0.5).astype(int)

print(classification_report(y_test, nn_predictions, digits=4))
print(confusion_matrix(y_test, nn_predictions))
```

The model outputs probabilities. A threshold of 0.5 converts them into class predictions. Later you can tune the threshold based on validation data if recall or false alarms are more important than the default threshold.

15. Compare Models Fairly

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd

def row(name, y_true, y_pred):
    return {
        "model": name,
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred),
        "recall": recall_score(y_true, y_pred),
        "f1": f1_score(y_true, y_pred)
    }

results = pd.DataFrame([
    row("Logistic Regression", y_test, logistic_predictions),
    row("Random Forest", y_test, rf_predictions),
    row("XGBoost", y_test, xgb_predictions),
    row("Neural Network", y_test, nn_predictions)
]).sort_values("f1", ascending=False)

print(results)
```

Select the best model using more than one metric. In intrusion detection, missing attacks can be costly, so recall matters. Too many false alarms are also harmful, so precision matters. F1 is a useful overall summary.

16. Understand the Confusion Matrix

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

ConfusionMatrixDisplay.from_predictions(
    y_test,
    rf_predictions,
    display_labels=["Normal", "Attack"]
)
plt.title("Random Forest Confusion Matrix")
plt.show()
```

The four cells represent true negatives, false positives, false negatives, and true positives. In a security setting, false negatives are especially important because they are attacks that the model missed.

17. Check for Overfitting

```
train_predictions = rf_model.predict(X_train)
test_predictions = rf_model.predict(X_test)

print("Train F1:", f1_score(y_train, train_predictions))
print("Test F1 :", f1_score(y_test, test_predictions))
```

If training performance is near-perfect but test performance is much worse, the model may be overfitting. Possible responses include limiting tree depth, tuning hyperparameters, collecting more varied data, reducing leakage, or using cross-validation.

18. Cross-Validation for More Reliable Estimates

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(
    RandomForestClassifier(
        n_estimators=150,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    ),
    X_train,
    y_train,
    cv=cv,
    scoring="f1",
    n_jobs=-1
)

print("F1 scores:", scores)
print("Mean F1:", scores.mean())
```

Cross-validation repeats training on multiple splits of the training data. It gives a more stable estimate than relying on one split while keeping the final test set untouched.

19. Save the Best Model

19.1 Scikit-learn / XGBoost model

```
import joblib

joblib.dump(rf_model, "models/cyberattack_model.pkl")
joblib.dump(list(X_train.columns), "models/feature_columns.pkl")
```

The feature-column list is saved because the inference API must provide features in the same order as training.

19.2 Save scaler and TensorFlow model

```
joblib.dump(scaler, "models/scaler.pkl")
nn_model.save("models/neural_network.keras")
```

Save every preprocessing object that is required at prediction time. A model without its preprocessing pipeline is incomplete.

20. Test Loading the Saved Model

```
import joblib

model = joblib.load("models/cyberattack_model.pkl")
feature_columns = joblib.load("models/feature_columns.pkl")

example = X_test.iloc[[0]][feature_columns]
prediction = model.predict(example)[0]
probability = model.predict_proba(example)[0, 1]

print("Prediction:", "Attack" if prediction == 1 else "Normal")
print("Attack probability:", probability)
```

Always test the saved model in a fresh process before building the API. This catches missing files and mismatched preprocessing early.

21. Upgrade to Multiclass Attack Classification

After binary classification works, keep the original attack labels and train a multiclass model.

```
from sklearn.preprocessing import LabelEncoder

multiclass_df = df.copy()
encoder = LabelEncoder()
y_multi = encoder.fit_transform(multiclass_df["label"].astype(str))

print(list(zip(encoder.classes_, encoder.transform(encoder.classes_))))
```

```
X_multi = multiclass_df.drop(columns=["label", "target"], errors="ignore")
X_multi = X_multi.select_dtypes(include=["number"])

X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X_multi, y_multi, test_size=0.20, random_state=42, stratify=y_multi
)

rf_multi = RandomForestClassifier(
    n_estimators=250,
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)
rf_multi.fit(X_train_m, y_train_m)
```

```
pred_m = rf_multi.predict(X_test_m)
print(classification_report(
    y_test_m,
    pred_m,
    target_names=encoder.classes_,
    digits=4
))
```

Multiclass evaluation should show per-class precision, recall, and F1 because some rare attacks may perform much worse than common attacks.

22. Build a FastAPI Prediction API

22.1 Basic backend file

```
# backend/main.py
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import pandas as pd

app = FastAPI(title="CyberSentinel AI API")

model = joblib.load("models/cyberattack_model.pkl")
feature_columns = joblib.load("models/feature_columns.pkl")

class PredictionRequest(BaseModel):
    features: dict[str, float]

@app.get("/")
def home():
    return {"message": "CyberSentinel AI API is running"}

@app.post("/predict")
def predict(request: PredictionRequest):
    row = pd.DataFrame([request.features])
    row = row.reindex(columns=feature_columns)

    if row.isna().any().any():
        return {"error": "Missing one or more required features"}

    pred = int(model.predict(row)[0])
    probability = float(model.predict_proba(row)[0, 1])

    return {
        "prediction": "Attack" if pred == 1 else "Normal",
        "attack_probability": round(probability, 4),
        "risk": (
            "HIGH" if probability >= 0.80
            else "MEDIUM" if probability >= 0.50
            else "LOW"
        )
    }
```

The API loads the model once when the server starts. Each request provides a dictionary of feature values. The code reorders them to match training columns, predicts the class, and returns a probability and simple risk category.

22.2 Run FastAPI

```
uvicorn backend.main:app --reload
```

Open the automatically generated `/docs` endpoint in your browser to test the API interactively during development.

23. Safer API Design

For a portfolio project, validate inputs carefully. Do not silently replace missing fields with arbitrary zeros unless that behavior is part of the training pipeline.

```
# Example additional validation idea
if not 0 <= request.features.get("destination_port", 0) <= 65535:
    return {"error": "destination_port must be between 0 and 65535"}
```

In a production-quality version, prefer a strongly typed Pydantic model with explicit fields rather than an unrestricted dictionary.

24. Build a Simple Streamlit Dashboard First

If React feels too large while you are still learning ML, build a small Streamlit dashboard first. You can replace it with React later.

```
pip install streamlit
```

```
# dashboard.py
import streamlit as st
import pandas as pd
import joblib

model = joblib.load("models/cyberattack_model.pkl")
features = joblib.load("models/feature_columns.pkl")

st.title("CyberSentinel AI")
st.write("Defensive network traffic classification demo")

values = {}
for feature in features:
    values[feature] = st.number_input(feature, value=0.0)

if st.button("Analyze traffic"):
    row = pd.DataFrame([values], columns=features)
    pred = model.predict(row)[0]
    prob = model.predict_proba(row)[0, 1]

    if pred == 1:
        st.error(f"Attack detected - probability {prob:.2%}")
    else:
        st.success(f"Traffic appears normal - attack probability {prob:.2%}")
```

```
streamlit run dashboard.py
```

This interface is enough to demonstrate model inference. Later you can build a richer React dashboard with charts, history, authentication, and an API.

25. Suggested React Dashboard Screens

- Overview: total analyzed flows, normal traffic, alerts, high-risk alerts.
- Recent threats table: time, predicted class, confidence, risk.
- Analytics: attacks over time and attack-type distribution.
- Model page: current model, evaluation metrics, confusion matrix, feature importance.
- Analyze page: controlled form or uploaded sample data for prediction.

26. Explainable AI with SHAP (Optional)

```
pip install shap
```

```
import shap

# TreeExplainer works well with many tree models.
explainer = shap.TreeExplainer(rf_model)
example_rows = X_test.sample(100, random_state=42)
shap_values = explainer.shap_values(example_rows)

# Exact plotting call can vary with SHAP/model versions.
# Start by inspecting the returned shape and SHAP documentation.
print(type(shap_values))
```

SHAP can help explain which features pushed a prediction toward normal or attack. Keep explanations separate from the raw model prediction so users understand they are interpretations, not absolute causal proofs.

27. Advanced Feature - TensorFlow Autoencoder Anomaly Detection

A supervised classifier learns known labels. An autoencoder can be trained primarily on normal traffic to learn a compressed representation. Traffic with unusually high reconstruction error can be flagged as anomalous.

27.1 Train on normal traffic

```
# Use the already fitted StandardScaler or create a dedicated scaler.
normal_train = X_train[y_train == 0]
normal_train_scaled = scaler.fit_transform(normal_train)

input_dim = normal_train_scaled.shape[1]

autoencoder = tf.keras.Sequential([
  tf.keras.layers.Input(shape=(input_dim,)),
  tf.keras.layers.Dense(64, activation="relu"),
  tf.keras.layers.Dense(16, activation="relu"),
  tf.keras.layers.Dense(64, activation="relu"),
  tf.keras.layers.Dense(input_dim, activation="linear")
```

```
])

autoencoder.compile(optimizer="adam", loss="mse")

autoencoder.fit(
    normal_train_scaled,
    normal_train_scaled,
    validation_split=0.20,
    epochs=30,
    batch_size=128,
    callbacks=[early_stopping],
    verbose=1
)
```

The input and target are the same. The network learns to reconstruct normal traffic. The bottleneck layer forces it to learn a compressed representation.

27.2 Calculate reconstruction error

```
X_test_scaled_ae = scaler.transform(X_test)
reconstructed = autoencoder.predict(X_test_scaled_ae)

reconstruction_error = np.mean(
    np.square(X_test_scaled_ae - reconstructed),
    axis=1
)

print(reconstruction_error[:10])
```

Normal traffic should generally reconstruct with lower error than unfamiliar or unusual patterns, although real datasets can overlap significantly.

27.3 Choose threshold using validation data

```
# Example only: choose the threshold on validation NORMAL data,
# not by looking at final test labels.
validation_normal = normal_train_scaled[-1000:]
reconstructed_val = autoencoder.predict(validation_normal)
val_error = np.mean(
    np.square(validation_normal - reconstructed_val),
    axis=1
)

threshold = np.percentile(val_error, 99)
print("Threshold:", threshold)
```

A percentile threshold is a starting point, not a universal rule. Evaluate false-positive and detection rates on held-out data before claiming good anomaly-detection performance.

28. Common Beginner Mistakes to Avoid

- Data leakage: allowing the label or a post-event feature to appear in the model inputs.
- Fitting scalers or encoders on all data before splitting.
- Reporting only accuracy on imbalanced data.
- Testing repeatedly on the final test set while tuning the model.

- Using a huge dataset before you know whether the pipeline works.
- Saving the model but forgetting the scaler, encoder, or feature order.
- Assuming a neural network must be better than Random Forest or XGBoost.
- Using live production traffic without authorization. Keep your portfolio work to public data and controlled environments.

29. Suggested Development Schedule

Phase	Goal	Deliverable
1	Environment + dataset	Project folder, venv, dataset loaded
2	Exploration + cleaning	Clean DataFrame and class statistics
3	Binary baseline	Logistic Regression metrics
4	Tree models	Random Forest + XGBoost comparison
5	Deep learning	TensorFlow network and learning curves
6	Model selection	Final comparison + saved artifacts
7	API	FastAPI /predict endpoint
8	UI	Streamlit or React dashboard
9	Advanced	Multiclass + SHAP + autoencoder
10	Portfolio polish	README, screenshots, architecture, demo

30. README Structure for GitHub

```
# CyberSentinel AI

## Overview
## Problem Statement
## Dataset
## Features
## Project Architecture
## Data Preprocessing
## Models Trained
## Evaluation Metrics
## Results
## API
## Dashboard
## Installation
## How to Run
## Limitations
## Future Improvements
## Ethical / Defensive Use
```

Include real metric tables and screenshots from your own run. Do not invent model accuracy numbers.

31. What to Say About the Project in an Interview

A strong explanation is: “I built a defensive network-intrusion classification system using labeled network-flow data. I cleaned and split the data carefully, trained Logistic Regression, Random Forest, XGBoost, and a TensorFlow neural network, and compared them using precision, recall, F1-score, and confusion matrices. I selected the best model based on held-out performance, saved the preprocessing and model artifacts, exposed inference through FastAPI, and created a dashboard for predictions and model statistics. I also explored anomaly detection using a TensorFlow autoencoder.”

32. CV Project Description

CyberSentinel AI - Machine Learning Intrusion Detection System

- Built a defensive network intrusion detection pipeline using Python, Pandas, Scikit-learn, XGBoost, and TensorFlow.
- Trained and evaluated Logistic Regression, Random Forest, XGBoost, and neural-network classifiers using precision, recall, F1-score, and confusion matrices.
- Deployed the selected model through FastAPI and designed a dashboard for traffic-risk predictions and threat analytics.
- Explored explainable AI and autoencoder-based anomaly detection for unusual network behavior.

33. Final Completion Checklist

- Project runs in a clean virtual environment.
- Dataset source and license/usage notes are documented.
- Binary classification works end to end.
- At least three models are trained and compared.
- Test data is not used for model tuning.
- Precision, recall, F1, and confusion matrix are reported.
- Best model and preprocessing artifacts are saved.
- Fresh-process model loading is tested.
- FastAPI endpoint returns predictions.
- Dashboard shows prediction and confidence/risk information.
- Multiclass attack detection is added after the binary version.
- Optional anomaly-detection experiment is clearly labeled as experimental.
- GitHub README explains setup, architecture, metrics, limitations, and ethical use.

34. Recommended Order for You to Start

Do the project in this exact order so the difficulty grows gradually:

1. Download one dataset and load it with Pandas.
2. Inspect labels, missing values, duplicates, and numeric features.
3. Create Normal=0 and Attack=1.

4. Split into training and test sets.
5. Train Logistic Regression as the baseline.
6. Train Random Forest and evaluate it.
7. Train XGBoost and compare it.
8. Add the TensorFlow neural network.
9. Choose and save the best model.
10. Build FastAPI, then the dashboard.
11. Only then add multiclass classification, explanations, and the autoencoder.

Best first milestone: Get Random Forest to train, predict, and print a confusion matrix on a clean binary dataset. Once that works, you already have the core ML pipeline.